LAPORAN PRAKTIKUM
PEMROGRAMAN BERBASIS FRAMEWORK

WEEK 4: Catch-All Routing, Optional Catch-All, Linking & Navigating pada Next.js

Disusun Oleh :
Ghoffar Abdul Ja'far 2341720035/TI3B

JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2025/2026

JOBSHEET PRAKTIKUM

Mata Kuliah : Pemograman Framework

Program Studi : D4 (Sarjana Terapan)

Topik Praktikum : Catch-All Routing, Optional Catch-All, Linking & Navigating pada Next.js Pages Router

Alokasi Waktu : 3 × 50 menit

A. **Tujuan Praktikum**

Setelah menyelesaikan praktikum ini, mahasiswa mampu:

1. Membuat catch-all route untuk menangkap banyak segmen URL.
2. Menggunakan optional catch-all route agar halaman tetap dapat diakses tanpa parameter.
3. Mengambil parameter URL berbentuk array menggunakan useRouter.
4. Menerapkan navigasi antar halaman menggunakan Link.
5. Melakukan navigasi imperatif menggunakan router.push().
6. Mengimplementasikan redirect sederhana berbasis kondisi (simulasi login).

B. **Dasar Teori Singkat**

1. **Segment & Slug pada URL**

- URL dapat terdiri dari beberapa segmen, contoh:

  /product/clothes/tops/t-shirt

- Setiap bagian dipisahkan oleh / dan disebut segmen.

2. Catch-All Route

Next.js memungkinkan menangkap semua segmen URL menggunakan:

- [...slug].js

Hasil parameter akan berbentuk array.

3. Optional Catch-All Route

Agar halaman tetap bisa diakses meskipun tanpa parameter:

- [[...slug]].js

4. Navigasi di Next.js

- Deklaratif: Link dari next/link
- Imperatif: router.push() dari next/router

C. Alat dan Bahan

Perangkat Lunak

- Node.js (LTS)
- NPM
- Visual Studio Code
- Browser (Chrome / Firefox)

Prasyarat

- Project Next.js Pages Router sudah tersedia
- Server dapat dijalankan (npm run dev)

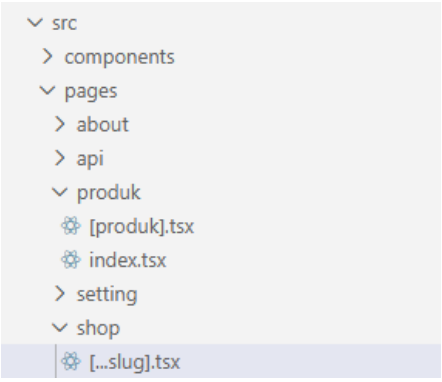
D. Langkah Kerja Praktikum

Langkah 1 – Menjalankan Project

1. Buka folder project Next.js.
2. Jalankan server:
3. npm run dev
4. Akses:
5. http://localhost:3000

Langkah 2 – Membuat Catch-All Route

1. Masuk ke folder pages.
2. Buat folder shop dan file [...slug].tsx:



3. Modifikasi Isi file [...slug].tsx dengan kode berikut:

```
src > pages > shop > [...slug].tsx > default
import { useRouter } from "next/router";

const HalamanToko = () => {
    const Router = useRouter();
    console.log(Router);
    // const { query } = useRouter();    Remove th
    return (
        <div>
            <h1>Halaman Toko</h1>
        </div>
    );
};

export default HalamanToko;
```

Cek menggunakan console.log apakah nilai segment berhasil didapat

- Jalankan browser dan ketik urlnya sebagai berikut

- Cek Vscode jika pada console.log dapat menampilkan nilai querynya berarti berhasil

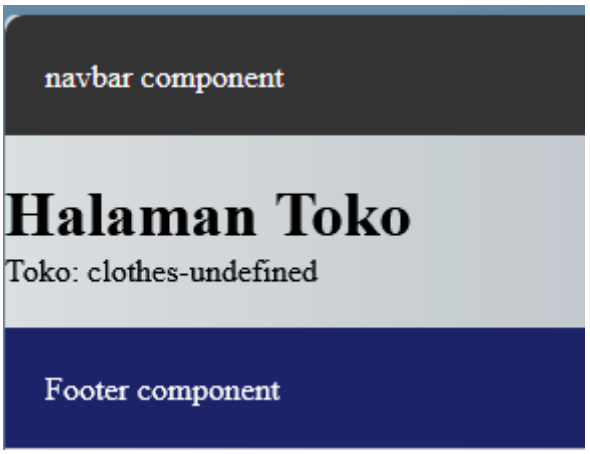
- Modifikasi [...slug].tsx untuk menampilkan nilai query

```
src > pages > shop > [...slug].tsx > ...
import { useRouter } from "next/router";

const HalamanToko = () => {
    // const Router = useRouter();    Remove this commented out code.
    // console.log(Router);
    const { query } = useRouter();
    return (
        <div>
            <h1>Halaman Toko</h1>
            <p>Toko: {query.slug && query.slug[0] + "-" + query.slug[1]}</p>
        </div>
    );
};

export default HalamanToko;
```

Langkah 3 – Pengujian Catch-All Route

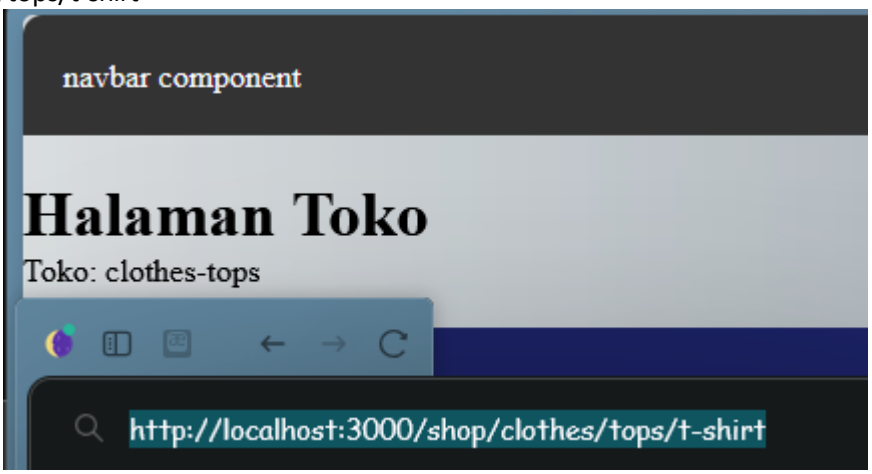
Akses URL berikut di browser:

/shop/clothes



/shop/clothes/tops

/shop/clothes/tops/t-shirt

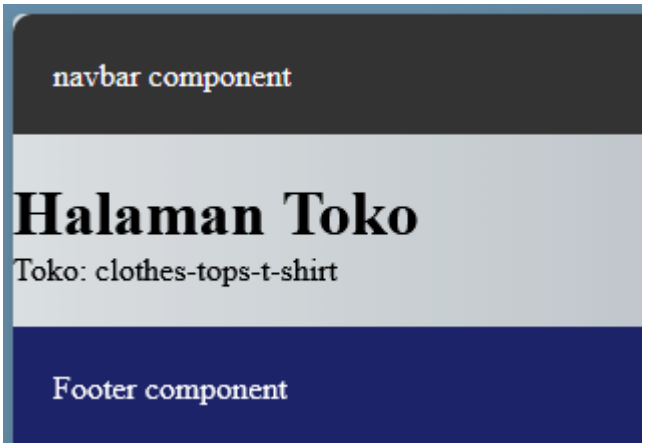


Jika dilihat ada yang terbaca undifined dan ada yang tidak terbaca ini dikarena segmennya dibatasi Cuma array[0] dan array[1]. Solusinya bagaimana ?

Modifikasi [...slug].tsx menjadi berikut

```
[...slug].tsx ×
praktikum-LinkNavigation > my-app > src > pages > shop > [...slug].tsx > ...
import { useRouter } from "next/router";


Windsurf: Refactor | Explain | Generate JSDoc | ×
const halamanToko = () => {
   // const Router = useRouter();
    //console.log(Router);
      const { query } = useRouter();
  return (
  <div>
     <h1>Halaman Toko</h1>
     {/* <p>Toko: {`${query.slug && query.slug[0]+"-"+ query.slug[1]}`}</p>  menggunakan backtick bukan petik satu */}
      <p>
          Toko: {Array.isArray(query.slug) ? query.slug.join("-") : query.slug}
      </p>

  </div>
     );
};

export default halamanToko;
```

Jalankan browser : **Berapapun banyaknya seqment tetap terbaca**



Untuk saat ini gunakan

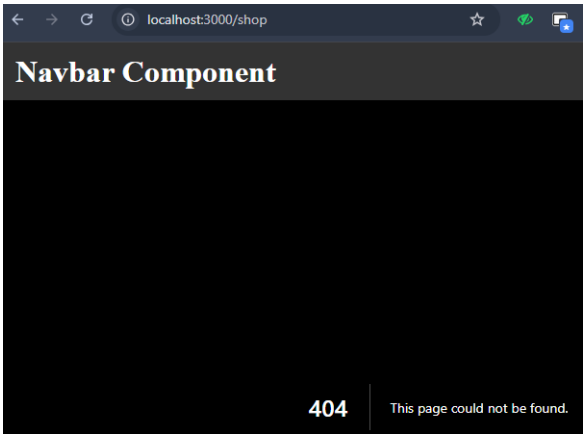
```
src > pages > shop > [...slug].tsx > HalamanToko
1   import { useRouter } from "next/router";
2
3   const HalamanToko = () => {
4       // const Router = useRouter();     Remove this commented out code.
5       // console.log(Router);
6       const { query } = useRouter();
7       return (
8           <div>
9               <h1>Halaman Toko</h1>
10              <p>Toko: {`${query.slug && query.slug[0]+"-"+query.slug[1]}`}</p>
11              {/* <p>Toko: {Array.isArray(query.slug) ? query.slug.join("-") : query.slug}</p> */}
12          </div>
13      );
14  };
15
16  export default HalamanToko;
```

Perhatikan bahwa:

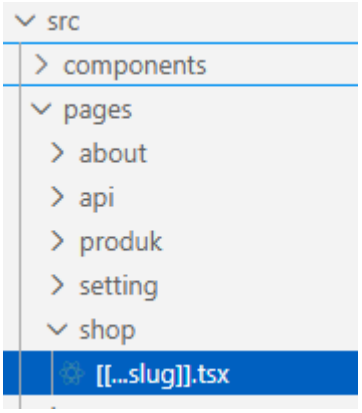
- slug berbentuk array
- Isi halaman berubah sesuai URL

Langkah 4 – Optional Catch-All Route

1. Jika menggunakan [...slug].js maka ketika mengakses shop akan terjadi error



2. Solusinya dengan Rename file: [...slug].js → [[...slug]].js



3. Sekarang akses:

/shop

4. Halaman dapat diakses meskipun tanpa parameter.

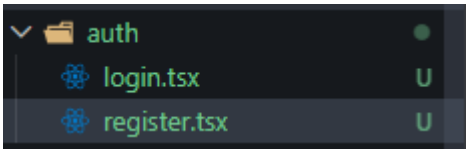
Langkah 5 – Validasi Parameter

Tambahkan validasi agar tidak error saat slug kosong:

```
<p>
  Kategori: {query.slug ? query.slug[0] : "Semua Kategori"}
</p>
```

Langkah 6 – Membuat Halaman Login & Register

1. Buat folder:
2. pages/auth
3. Buat file:
   - login.jsx
   - register.jsx



Modifikasi login.jsx:

```
src > pages > auth > login.tsx > ...
import Link from "next/link";

const HalamanLogin = () => {
  return (
    <div>
      <h1>Halaman Login</h1>
      <Link href="/auth/register">Ke Halaman Register</Link>
    </div>
  );
};

export default HalamanLogin;
```

Modifikasi register.jsx:

```
src > pages > auth > register.tsx > ...
import Link from "next/link";

const HalamanRegister = () => {
  return (
    <div>
      <h1>Halaman Register</h1>
      <Link href="/auth/login">Ke Halaman Login</Link>
    </div>
  );
};

export default HalamanRegister;
```

Langkah 7 – Navigasi Imperatif (router.push)

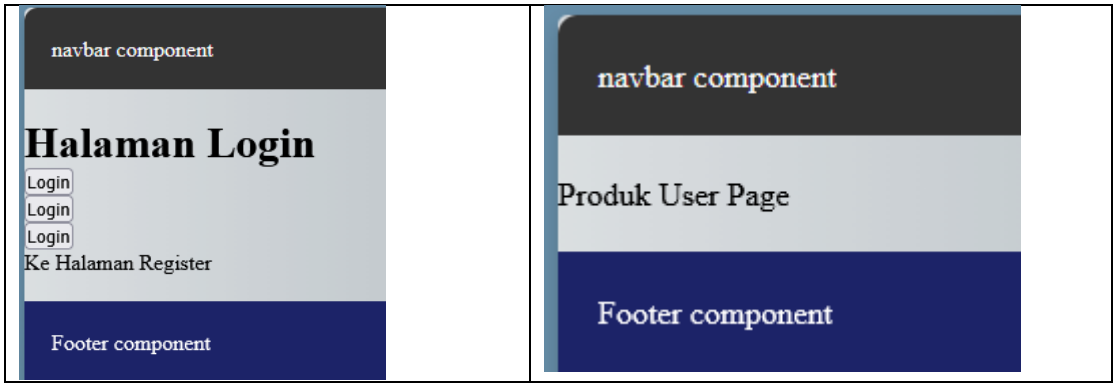
1. Tambahkan button login:

```
login.tsx U ×   register.tsx U   index.tsx 4, M   [[...slug]].tsx 1, U
src > pages > auth > login.tsx > ...
import Link from "next/link";
import { useRouter } from "next/router";

const HalamanLogin = () => {
    const{ push } = useRouter();
    const handlerLogin = () => {
      push('/produk');
    }
  return (
    <div>
      <h1>Halaman Login</h1>
      <button onClick={handlerLogin}>Login</button> <br />
      <button onClick={() => push('/produk')}>Login</button> <br />
      <button onClick={() => handlerLogin()}>Login</button> <br />
      <Link href="/auth/register">Ke Halaman Register</Link>
    </div>
  );
};

export default HalamanLogin;
```

Kode	Cara Kerja	Kapan Dipanggil	Kelebihan	Kekurangan	Rekomendasi
onClick={handlerLogin}	Mengirim referensi fungsi ke event handler	Saat tombol diklik	Paling bersih, efisien, dan best practice	Tidak bisa kirim parameter langsung	**Sangat direkomendasikan**
onClick={() => push('/produk')}	Arrow function memanggil fungsi di dalamnya	Saat tombol diklik	Praktis untuk aksi sederhana	Kurang reusable jika logika bertambah	Direkomendasikan untuk navigasi sederhana
onClick={() => handlerLogin()}	Arrow function membungkus pemanggilan fungsi	Saat tombol diklik	Fleksibel untuk kirim parameter	Redundant jika tanpa parameter	Direkomendasikan hanya jika perlu argumen
onClick={handlerLogin()}	Fungsi langsung dieksekusi	Saat render	Tidak ada	Bug: fungsi tidak menunggu klik	**Tidak direkomendasikan**

Saat ini gunakan **onClick={() => handlerLogin()}**

2. Klik tombol dan perhatikan perpindahan halaman tanpa reload.
3. Jika di klik button login maka akan menuju /produk



Note :

- Pastikan code untuk redirect tidak aktif , jika aktif maka ketika masuk ke produk akan lansung redirect ke login

```
6.praktikum-ApiRoutes > my-app > src > pages > produk > index.tsx > ...
import { useRouter } from "next/router";
import { useEffect, useState } from "react";

Windsurf: Refactor | Explain | Generate JSDoc | X
const produk = () => {
  // const [isLogin, setIsLogin] = useState(false);
  // const { push } = useRouter();
  // useEffect(() => {
  //   if (!isLogin) {
  //     push("/auth/login");
  //   }
  //   },[]);
  return (
  <div>Produk User Page</div>
  );
};

export default produk;
```

Langkah 8 – Simulasi Redirect (Belum Login)

1. Di halaman product, pada index.tsx tambahkan beberapa code berikut:

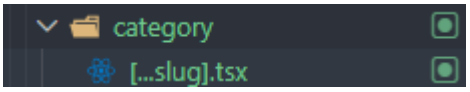
```
src > pages > produk > index.tsx > default
import { useRouter } from "next/router";
import { useEffect, useState } from "react";

const produk = () => {
    const [isLogin, setIsLogin] = useState(false);    React Hook "useState" is called in
    const { push } = useRouter();    React Hook "useRouter" is called in function "produ
    useEffect(() => {    React Hook "useEffect" is called in function "produk" that is n
        if (!isLogin) {
            push('/auth/login');
        }
    }, []);
    return (
        <div>
            Produk User Page
        </div>
    );
};

export default produk;
```

2. Jika Akses /product → otomatis diarahkan ke login.

E. Tugas Praktikum

Tugas 1 (Wajib)

- Buat catch-all route:



- /category/[...slug].tsx

```
src > pages > category > [...slug].tsx > CategoryPage
import { useRouter } from "next/router";

const CategoryPage = () => {
  const { query } = useRouter();
  const slugList = Array.isArray(query.slug) ? query.slug : [];

  return (
    <div>
      <h1>Halaman Category</h1>
      {slugList.length > 0 ? (
        <ul>
          {slugList.map((segment, index) => (
            <li key={`${segment}-${index}`}>{segment}</li>
          ))}
        </ul>
      ) : (
        <p>Parameter category belum tersedia.</p>
      )}
    </div>
  );
};

export default CategoryPage;
```

- Tampilkan seluruh parameter URL dalam bentuk list.

Tugas 2 (Wajib)

- Buat navigasi:
  - Login → Product (imperatif)

```
{/* Navigasi Imperatif */}
<button onClick={handlerLogin}>Login</button> <br />
```

  - Login ↔ Register (Link)

```
{/* Navigasi Link */}
<Link href="/auth/register">Ke Halaman Register</Link>
```

Tugas 3 (Pengayaan)

- Terapkan redirect otomatis ke login jika user belum login.

```
src > pages > produk > index.tsx > default
import { useRouter } from "next/router";
import { useEffect, useState } from "react";

const produk = () => {
    const [isLogin, setIsLogin] = useState(false);    React Hook "us
    const { push } = useRouter();    React Hook "useRouter" is calle
    useEffect(() => {    React Hook "useEffect" is called in functio
        if (!isLogin) {
            push('/auth/login');
        }
    }, []);
    return (
        <div>
            Produk User Page
        </div>
    );
};

export default produk;
```

F. Pertanyaan Evaluasi

1. Apa perbedaan [id].js dan [...slug].js?

   = [id] satu level, [...slug] menangkap banyak segmen URL sekaligus.

2. Mengapa slug berbentuk array?

   = Karena menangkap beberapa hierarki segmen URL dalam satu variabel.

3. Kapan sebaiknya menggunakan Link dan router.push()?

   = Link untuk navigasi komponen, router.push untuk navigasi lewat logika.

4. Mengapa navigasi Next.js tidak me-refresh halaman?

   = Menggunakan client-side routing yang hanya memperbarui konten yang berubah.